

SENIOR SWE INTERVIEW SERIES

Module 3: UDP

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 3 — UDP

UDP questions test judgment, not memorization: when do you drop TCP's guarantees, and what do you rebuild in the application? This is also the gateway to QUIC (Module 4).

Topic 3.1 — UDP Fundamentals

1. Why Interviewers Ask This

“TCP vs UDP” is the most common warm-up in networking rounds — but at the senior level, the follow-ups go straight to header contents, socket behavior differences, and what “connectionless” means for load balancing and NAT.

2. Core Concept

UDP = IP + ports + optional checksum. 8-byte header: src port, dst port, length, checksum. No connections, no ordering, no reliability, no flow/congestion control, no handshake. One `sendto()` = one datagram = (usually) one IP packet. Message boundaries are preserved — unlike TCP's byte stream.

3. Internal Working

- No connection state: a single UDP socket receives datagrams from *any* peer (`recvfrom` returns the source address). `connect()` on a UDP socket only fixes the default destination and filters inbound — no packets are exchanged.
- Demux is by (dst IP, dst port) only — this is why one DNS server socket serves millions of clients.
- Datagrams larger than the socket receive buffer's free space are **dropped silently** (counter: `netstat -su` “receive buffer errors”). Datagrams above path MTU are fragmented at the IP layer (dangerous — see below).

4. Packet Flow Explanation

```
app sendto(sock, 900B, addr)
-> kernel prepends UDP hdr (8B) + IP hdr (20B) -> one 928B packet -> wire
no ACK expected, no retransmit timer, no state kept.
receiver: packet -> UDP checksum verify -> socket recv buffer -> recvfrom()
buffer full? drop. no listener? ICMP port-unreachable back.
```

Compare TCP: 3 packets before data, ACK for everything, state on both ends.

5. ASCII Diagram

```

UDP header (8 bytes):
+-----+-----+
|  src port   |  dst port   |
+-----+-----+
|  length     |  checksum   |
+-----+-----+
|                payload...  |
+-----+-----+

TCP: [stream, 20-60B hdr, connected, reliable, ordered, paced]
UDP: [datagrams, 8B hdr, fire-and-forget, app owns everything else]

```

6. Real Production Example

DNS (queries), QUIC/HTTP3 (all of Google/Meta/Cloudflare edge traffic), video conferencing (Zoom/Meet — RTP over UDP), game servers, syslog, StatsD/metrics fire-and-forget, WireGuard VPN, NTP.

7. Advantages

- Zero-RTT to first byte; no handshake, no state.
- No head-of-line blocking; loss of one datagram doesn't delay others.
- Multicast/broadcast possible; TCP is strictly point-to-point.
- App controls latency/reliability trade-off precisely.

8. Trade-offs

- You inherit every hard problem TCP solved: loss, reordering, duplication, congestion, MTU discovery.
- No congestion control = you can melt networks (and get throttled/blocked by ISPs).
- NATs/firewalls handle UDP worse: short idle timeouts (~30–120s), some networks block it (QUIC always keeps a TCP fallback for this).
- Spoofable source addresses → amplification attacks (DNS/NTP reflection).

9. Common Mistakes

- “UDP is faster than TCP” stated flatly — per-packet it's lighter and has no HoL blocking, but with loss and no recovery, *goodput* can be worse. Precision here signals seniority.
- Assuming a UDP datagram maps to exactly one packet regardless of size (a 60KB datagram becomes ~40 IP fragments; one lost fragment kills the whole datagram).
- Forgetting UDP has checksums (optional in IPv4, mandatory in IPv6).

10. Performance Impact

No handshake: first byte arrives 1 RTT earlier than TCP (2–3 RTTs earlier vs TCP+TLS1.2). Syscall-per-datagram overhead is the real cost at high packet rates — hence `sendmmsg / recvmmsg`, GSO for UDP, and kernel-bypass (DPDK) in trading systems.

11. Common Interview Questions

1. What exactly is in a UDP header, and what does its absence of fields imply?

2. What happens when a UDP datagram is lost / arrives out of order / is duplicated? (Nothing — app's problem.)
3. Can UDP datagrams be fragmented, and why is that bad?

12. Follow-up Questions

- “What does `connect()` on a UDP socket do?” → sets default peer + filters inbound + lets you receive ICMP errors; no packets sent.
- “How does a NAT track UDP without connections?” → address/port mapping with idle timer; that's why STUN/keep-alives exist.

13. Debugging Scenarios

- Silent metric loss: `netstat -su` → “packet receive errors / receive buffer errors” → raise `net.core.rmem_max` + app read faster.
- UDP service reachable locally, not remotely → firewalls block or NAT expired; check with `nc -u`, `tcpdump` both ends.

14. Best Practices

- Keep datagrams $\leq \sim 1400\text{B}$ (or 1200B, QUIC's choice) to dodge fragmentation.
- Size receive buffers for burst rate; monitor drop counters — UDP drops are silent by design.
- Always rate-limit/validate: never build a UDP responder that answers big to small unauthenticated queries (amplification).

15. Practice Questions

1. Metrics pipeline loses 2% of StatsD packets only during deploys. Hypothesize. (Burst > socket buffer during restart-induced CPU contention; raise `rmem`, batch metrics.)
2. Why can one UDP socket serve 1M clients while TCP needs 1M sockets? (Demux by dst only vs 4-tuple; no per-peer state.)

Topic 3.2 — When UDP Is Better Than TCP

1. Why Interviewers Ask This

This is a *judgment* question. System design rounds at Netflix/Uber/Zoom-type companies hinge on choosing the right transport for video, telemetry, gaming, and RPC — and defending it.

2. Core Concept

Choose UDP when **at least one** of these holds: 1. **Stale data is worthless** (live audio/video/game state — a retransmitted old frame is useless; skip it). 2. **Latency beats completeness** (real-time > reliable). 3. **You'll build your own reliability better tailored than TCP's** (QUIC, RTP+FEC, reliable-UDP game protocols). 4. **Request/response is tiny** (DNS: handshake would triple cost). 5. **Multicast/broadcast needed**.

3. Internal Working

The key insight interviewers want: TCP couples four guarantees (reliability, ordering, congestion control, stream abstraction) — you can't turn any off. UDP lets you unbundle: e.g., a video call wants congestion awareness (yes) + ordering (per-frame, not global) + reliability (no — use FEC/concealment). TCP's total ordering forces head-of-line blocking that real-time apps cannot accept.

4. Packet Flow Explanation

Live video over TCP vs UDP when packet #2 (frame 2) is lost:

```
TCP: frame1 [frame2 LOST] frame3 frame4 ...
      receiver kernel BUFFERS frames 3,4 (can't deliver out of order)
      app sees NOTHING until frame2 retransmitted (>=1 RTT, maybe 200ms RTT)
      result: freeze, then fast-forward burst. Latency accumulates forever.
UDP: frame2 lost -> app shows frame1 again or conceals; frames 3,4 play
      on time. One glitch, zero added latency.
```

5. ASCII Diagram

```

              need every byte, order matters?
              yes                               no
              |                               |
          latency-critical?                 real-time media/state?
              no                             yes
              |                             |
          TCP                               UDP + FEC/concealment
    (files, APIs, DBs)                   (calls, games, telemetry
                                         where newest > complete)
          QUIC (or custom reliable
          UDP: fintech, gaming)

```

6. Real Production Example

- **Zoom/Meet/Teams:** UDP (RTP/SRTP) with FEC and jitter buffers; they fall back to TCP only when UDP is blocked — and quality visibly degrades.
- **HTTP/3:** Google moved the web to UDP to escape TCP's HoL blocking and kernel ossification.
- **Games** (Riot, Valve): custom reliable-UDP with per-channel reliability (position updates unreliable, chat reliable).
- **DNS:** query fits one packet; TCP only for large answers/zone transfers (and DoT/DoH).

7. Advantages (in these scenarios)

- Constant, low latency under loss; no retransmission-induced freezes.
- Per-message control (some messages reliable, some not — impossible on one TCP stream).
- Faster connection setup; better for short-lived exchanges.

8. Trade-offs

- Engineering cost: jitter buffers, FEC, sequencing, congestion control are hard to get right.
- Ops cost: UDP-hostile networks require TCP/443 fallback paths → two stacks to maintain.
- Fairness responsibility: without congestion control you harm co-located traffic (and your own).

9. Common Mistakes

- Choosing UDP for a “fast API” — request/response APIs need reliability; you’d rebuild TCP badly. (Right answer for fast APIs: connection reuse + QUIC/HTTP3, not raw UDP.)
- Forgetting the fallback: enterprise networks block UDP; a UDP-only product fails in exactly the customers who pay most.
- Claiming “UDP has no congestion control so it’s faster” as a *benefit* without acknowledging the ecosystem damage + mandatory CC in anything serious (QUIC has full CC).

10. Performance Impact

For a 30ms-RTT call with 1% loss: TCP adds ≥ 30 –200ms stalls per loss event and latency compounds (jitter buffer must grow); UDP+FEC holds end-to-end latency at $\sim \text{encode} + \text{network} + \text{jitter}$ (~ 100 ms) with graceful quality loss. For DNS: UDP = 1 RTT total; TCP would be 2 RTTs (+TLS more).

11. Common Interview Questions

1. Design video conferencing — which transport, why, what do you add on top?
2. Why does DNS use UDP (and when does it switch to TCP)? (Fits one datagram; TCP for >512B/1232B EDNS truncation-fallback, zone transfers.)
3. Would you use UDP for a payments API? (No — and articulate why.)

12. Follow-up Questions

- “How do you keep a UDP media stream fair to TCP flows?” → implement CC (e.g., GCC in WebRTC, QUIC’s CUBIC/BBR).
- “Network blocks UDP — what does your app do?” → detect quickly, fall back to TCP/TLS 443, telemetry on fallback rate.

13. Debugging Scenarios

- Users on corporate networks report terrible call quality: check transport telemetry — they’re on TCP fallback; fix = TURN/443 relay or QUIC-looking fallback.
- Game feels laggy though ping is low: loss on the unreliable channel; add FEC or adaptive send rate.

14. Best Practices

- Decide reliability *per message type*, not per app.
- Ship with congestion control and TCP fallback from day one.
- Measure “freeze time”/staleness, not just packet loss — the user-facing metric.

15. Practice Questions

1. Telemetry from 1M IoT devices, 1 reading/min, 40B each. TCP or UDP? Defend both sides, then pick. (UDP with app-level ack for config commands; occasional loss acceptable, connection state $\times 1$ M too costly — or MQTT-over-TCP if delivery matters. The *reasoning* is the answer.)
2. Stock ticker fan-out to 10k trading clients on a LAN: what does UDP multicast buy you that TCP can’t do at all? (One packet serves all subscribers; per-client TCP = $10k \times$ bandwidth + serialization skew.)

Topic 3.3 — Reliability Strategies over UDP

1. Why Interviewers Ask This

“Design reliable transfer over UDP” is a full-blown interview question at Google/Amazon — effectively “re-derive TCP, justify every piece.” It tests protocol design ability directly.

2. Core Concept

The toolbox, in increasing sophistication: 1. **Sequence numbers** — detect loss, reordering, duplication (the foundation; nothing works without them). 2. **ACK + retransmit** (ARQ): stop-and-wait → sliding window → selective ACK. 3. **Timeouts** with RTT estimation (else you retransmit too early/late). 4. **FEC** (forward error correction): send k data + m parity packets; receiver reconstructs any k of $k+m$ — repairs loss with *zero* extra RTT, at fixed bandwidth cost. 5. **NACK-based repair**: receiver reports gaps (efficient at scale for streams; sender needn't track per-receiver state). 6. **Congestion control** — mandatory for anything shipping at volume. 7. **Idempotency/dedup keys** at the semantic level.

3. Internal Working

A production-grade design (this is what interviewers want on the whiteboard): - Header: `{conn_id, seq, ack, sack_bitmap, timestamp}`. - Sender: sliding window of unACKed packets; per-packet send time; $RTO = SRTT + 4 \cdot RTT_{VAR}$; on SACK gap → selective retransmit; on timeout → backoff + window reduction (AIMD or copy CUBIC). - Receiver: reorder buffer; cumulative ack + bitmap of received-out-of-order; deliver in order (reliable mode) or immediately (unreliable-sequenced mode: drop anything older than newest delivered). - `conn_id` instead of 4-tuple → survives NAT rebinding/client IP change (QUIC's connection migration trick — mentioning this is a strong signal).

4. Packet Flow Explanation

```
send: [seq=1][seq=2][seq=3][seq=4][seq=5]      (window=5)
loss: seq=3 dropped
recv ACKs: ack=1 ... ack=2, bitmap{4,5 received}
sender: gap at 3 detected after bitmap shows 4,5 -> retransmit ONLY 3
recv: 3 arrives -> ack=5 -> window slides
FEC alternative: send  $P = 1^2 3^4 5$  (XOR parity). Receiver rebuilds 3
                  from {1,2,4,5,P}. No retransmit, no extra RTT.
```

5. ASCII Diagram

```
Reliability spectrum (pick per message class):
fire-and-forget < sequenced-unreliable < FEC-protected < NACK-repair
(metrics)          (game position)          (live video)      (video, VoD)
                                     < ACK+retransmit reliable+ordered
                                     (chat, control msgs) = ~TCP
Latency cost:  0          0          0 (+bw)          ~1 RTT    1+ RTT
```

6. Real Production Example

- **QUIC**: the fully-worked example — seq numbers never reused (retransmits get new packet numbers, killing TCP's retransmission ambiguity), SACK ranges, pacing, CUBIC/BBR.
- **WebRTC**: RTP seq + NACK (RTX) + FEC (flexfec) + jitter buffer, congestion control (transport-cc/GCC).

- **SRT** (broadcast video), **KCP** (gaming, ~30–40% faster than TCP on lossy links at 10–20% bandwidth premium), Aeron (trading).

7. Advantages

- Tailored trade-offs per message class; TCP forces one-size-fits-all.
- FEC gives zero-RTT repair TCP fundamentally cannot offer.
- New packet numbers on retransmit → unambiguous RTT samples (fixes Karn's problem).

8. Trade-offs

- You will spend months on edge cases TCP solved over 40 years (RTT estimation, reordering heuristics, CC fairness).
- FEC costs constant bandwidth even at zero loss; ARQ costs latency only on loss — hybrid (FEC for base, NACK for burst) is the pro answer.
- Userspace processing cost (no kernel offloads) — QUIC burns ~2–3× TCP's CPU per byte (improving with UDP GSO/kTLS-style offloads).

9. Common Mistakes

- Designing ACK-per-packet stop-and-wait (throughput = 1 pkt/RTT — always call out sliding window).
- No congestion control ("it's internal traffic" — until it isn't).
- Reusing sequence numbers for retransmits (ambiguous RTT, the exact bug QUIC fixed).
- Forgetting dedup: retransmits create duplicates; consumers must be idempotent or dedup by seq.

10. Performance Impact

FEC at $k=10, m=2$: +20% bandwidth, repairs ≤ 2 losses/group with 0 added RTT. ARQ: repair = RTT (NACK) to RTT+RTO (timeout). On a 150ms path, that's the difference between invisible and a visible freeze. Hybrid schemes dominate production media stacks.

11. Common Interview Questions

1. Design reliable file transfer over UDP (the classic — draw header, window, timers).
2. How would you handle reordering vs loss distinction? (Time-based like RACK, or small reorder tolerance window.)
3. FEC vs retransmission — when each?

12. Follow-up Questions

- "How do you pick RTO?" → EWMA SRTT + variance, Karn's rule or QUIC-style unique packet numbers.
- "How does the receiver signal loss efficiently for 1M receivers?" → NACK aggregation/multicast repair or FEC-only (no feedback path).
- "What about flow control?" → receiver-advertised window/credits, same as TCP conceptually.

13. Debugging Scenarios

- Custom protocol throughput collapses at 0.5% loss → likely stop-and-wait or fixed huge RTO; instrument RTT estimator.

- Duplicates observed downstream → retransmit + missing dedup; add seq-based dedup window.

14. Best Practices

- Steal QUIC's ideas: monotonically increasing packet numbers, ACK ranges, explicit ECN support, pacing.
- Simulate loss/reorder/dup in tests (`tc netem loss 2% delay 50ms reorder 5%`) — protocols that only met clean networks die in production.

15. Practice Questions

1. Sketch a header for a game protocol with 3 channels: reliable-ordered (chat), unreliable-sequenced (positions), reliable-unordered (item pickups). What per-channel state does each end keep?
2. $k=20$ FEC group, parity $m=?$, target: survive 5% random loss with $<0.1\%$ unrecovered groups. Reason about m . (Binomial tail: $P(>m \text{ losses in } 20+m) — m=3-4$ lands near target; exact math less important than the approach.)

Topic 3.4 — Production Use Cases

1. Why Interviewers Ask This

Senior candidates are expected to map protocol theory to real systems they'd operate — including the unglamorous parts: NAT traversal, buffer tuning, DDoS exposure.

2. Core Concept

The big five production UDP families: 1. **DNS** — tiny request/response; UDP:53 (Module 5). 2. **QUIC / HTTP/3** — the modern web transport; UDP:443 (Module 4). 3. **Real-time media** — RTP/SRTP for calls, WebRTC; latency-first. 4. **VPN/tunnels** — WireGuard, IPsec NAT-T, VXLAN, GTP (mobile core!) — encapsulation over UDP traverses NAT and hashes well for ECMP. 5. **Telemetry/gaming/trading** — StatsD, syslog, game state, market data multicast.

3. Internal Working

Why tunnels standardized on UDP rather than raw IP protocols: NATs and ECMP routers understand UDP ports. A VXLAN packet's *source port* is set to a hash of the inner flow → core routers spread tunnel traffic across paths without parsing the tunnel. This detail (entropy in source port) is a beloved infra-interview nugget.

4. Packet Flow Explanation

NAT traversal for a call (WebRTC/STUN/ICE) — the flow interviewers ask about:

1. A → STUN server (UDP): "what's my public ip:port?" → gets mapping
2. B does the same. A,B exchange candidates via signaling (any channel).
3. Both fire UDP probes at each other's public mapping simultaneously → each side's outbound packet opens its own NAT pinhole ("hole punch")
4. Direct P2P UDP flows. If both NATs are symmetric → relay via TURN.

5. ASCII Diagram

```
A -----> NAT-A ===== internet ===== NAT-B <----- B
|outbound opens pinhole|                |pinhole opened by B|
A's packets arrive at NAT-B's pinhole => delivered to B
fallback: A --> TURN relay <-- B    (both connect outbound; relay copies)
```

6. Real Production Example

- **Cloudflare/Google edges**: >30% of traffic already HTTP/3-over-UDP.
- **Zoom**: UDP 8801; TURN/TCP fallback tiering documented publicly.
- **WireGuard**: entire modern VPN industry (Tailscale) = UDP + NAT traversal exactly as above.
- **Mobile networks**: every packet your phone sends rides GTP-over-UDP inside the carrier core.
- **Finance**: exchange market data is UDP multicast with sequence-numbered gap-fill (retransmission request channels).

7. Advantages

- One socket, millions of peers; NAT/ECMP friendliness; multicast; latency floor.

8. Trade-offs / Operational Risks

- **DDoS**: UDP spoofing → reflection/amplification (DNS 50×, NTP 500×, memcached 50,000×). Any UDP service must consider being both victim and reflector. Mitigations: response ≤ request until source validated (QUIC's anti-amplification 3× rule, DNS cookies), BCP38 egress filtering.
- Stateful middlebox timeouts (~30s common for UDP!) → keep-alives every ~15–25s (WireGuard's persistent-keepalive=25).
- No kernel backpressure: apps must implement their own or drop.

9. Common Mistakes

- Deploying a UDP service without an amplification review.
- Assuming UDP “just works” through corporate networks — always ship a TCP/443 fallback and measure fallback rates.
- Using one socket + one thread for 10 Gbps of datagrams (need SO_REUSEPORT sharding, recvmmsg, GRO).

10. Performance Impact

Well-tuned UDP stacks reach millions of pps/core with batching + GSO/GRO; naive one-syscall-per-packet caps around ~300–500k pps/core. QUIC CPU cost ≈ 2× TCP+TLS today — a real capacity-planning input at CDN scale.

11. Common Interview Questions

1. How does a video call connect two laptops that are both behind NAT?
2. Why do VPNs and VXLAN run over UDP instead of their own IP protocol numbers?
3. What is a UDP amplification attack; how do you avoid building a reflector?

12. Follow-up Questions

- “What’s a symmetric NAT and why does it break hole punching?” → mapping differs per destination → predicted pinhole wrong → TURN relay required.
- “How does QUIC prevent amplification before the handshake completes?” → server sends $\leq 3\times$ bytes received until client address is validated.

13. Debugging Scenarios

- Calls connect on home WiFi, fail on hotel WiFi → UDP blocked; verify by forcing TURN/TCP; monitor ICE candidate-pair stats.
- VPN drops every ~30s idle → NAT UDP timeout; set persistent keepalive 25s.
- Market-data gaps at market open → socket buffer overflow at burst; `netstat -su`, raise rmem, dedicate cores.

14. Best Practices

- Keep-alive interval < smallest NAT UDP timeout on path (25s is the industry folk constant).
- `SO_REUSEPORT` + per-core sockets for high-rate UDP services.
- Rate-limit responses, require proof-of-source (cookies) before sending large replies.

15. Practice Questions

1. Design the transport for a multiplayer shooter: tick rate 64Hz, 200 players/match, worldwide. (UDP; unreliable-sequenced state snapshots + delta encoding; reliable channel for events; regional relays; lag compensation; TCP/443 fallback for hostile NATs.)
2. Your DNS resolver fleet is being used in a reflection attack. What do you change without breaking legitimate clients? (Response rate limiting per prefix, minimal-ANY answers, require TCP fallback via TC bit for suspicious sources, DNS cookies.)

MODULE 3 — One-Page Cheat Sheet

UDP HEADER	8B: src port dst port length checksum. That's all.
SEMANTICS	datagrams (boundaries kept), no order/reliability/CC/handshake one socket serves all peers (demux on dst only)
WHEN UDP	stale-data-worthless (live media, game state) tiny req/resp (DNS) custom reliability (QUIC) multicast tunnels
WHEN TCP	must-have-every-byte + no extreme latency need: APIs, files, DBs
RELIABILITY	seq numbers -> sliding window + SACK -> RTT-based RT0 ->
TOOLBOX	FEC (0-RTT repair, +bw) NACK (scale) CC (mandatory) dedup
KEY DESIGNS	QUIC: new pkt# on retransmit, ACK ranges, conn-id migration, 3x anti-amplification. WebRTC: RTP+NACK+FEC+jitter buffer+GCC
NAT REALITY	UDP idle timeout ~30-120s -> keepalive ~25s hole punching via STUN; symmetric NAT -> TURN relay
DANGER	amplification (DNS 50x, memcached 50000x) -> validate source, response<=request, rate limit fragmentation: keep <=1400B
OPS	silent drops: netstat -su rmem tuning sendmmsg/GSO for pps
FALLBACK	always ship TCP/443 fallback; measure fallback rate

MODULE 3 — Top Interview Questions

1. TCP vs UDP — beyond the checklist: what does each header field's absence in UDP force the app to do?
2. Design reliable file transfer over UDP. (Sliding window, SACK bitmap, RTT-adaptive RTO, CC, MTU-safe chunks.)
3. Why is UDP the right base for QUIC/HTTP3 rather than a new IP protocol? (NAT/firewall traversal, no kernel/middlebox ossification, userspace iteration.)
4. Video call freezes on TCP fallback — explain the mechanism. (Retransmission + total ordering = HoL blocking + growing jitter buffer.)
5. How does hole punching work; when does it fail?
6. FEC vs ARQ: derive when each wins (RTT vs bandwidth cost).
7. What makes UDP services DDoS reflectors and how do you design against it?

MODULE 3 — Common Mistakes

- “UDP is faster” without qualifying goodput-under-loss and CC responsibilities.
- Designing stop-and-wait; reusing seq numbers on retransmit.
- No TCP fallback; no keep-alives through NAT; datagrams > MTU.
- Ignoring silent socket-buffer drops (the #1 real-world UDP bug).
- Building an amplification reflector.

MODULE 3 — Mock Interview (10 min)

Q1. “Build a real-time multiplayer game transport. Walk me through it.” *Strong answer:* UDP; channels with per-class reliability (unreliable-sequenced snapshots @64Hz with delta+interpolation, reliable-ordered control); seq+ack bitmap header; adaptive send rate (CC!); client prediction + server reconciliation for perceived latency; STUN/relay infrastructure; hostile-network fallback; loss/jitter simulation in CI.

Q2. “Your custom UDP protocol works in staging, but throughput collapses for Australian users.” *Strong answer:* long-RTT exposes protocol flaws: fixed RTO (too aggressive/passive), window too small ($\text{throughput} = \text{window} / \text{RTT}$), no pacing causing burst loss; verify with RTT-scaled testbed (`tc netem delay 150ms loss 0.5%`); fix adaptive RTO + BDP-sized window + pacing.

Q3. “Why does WhatsApp calling work behind almost any NAT but your prototype doesn’t?” *Strong answer:* full ICE ladder: host → STUN srflx → TURN relay (UDP → TCP → TLS/443); simultaneous open keep-alives; they invest in relays near users — reachability is an infrastructure product, not a protocol trick.